

# Practical Example

useCallback + useMemo + React.memo Working Together in a Shopping Cart

Simple · Testable · With comments on what breaks without them

- Purpose See all 3 hooks working together in one real example
- Testing Each section shows: what to console.log, what to expect, what breaks
- Series React Interview Prep — Companion to Topics 7, 8 & 9

by Akshay Chavhan • React Interview Series

## The Scenario — Shopping Cart App

A simple cart with products and a theme toggle. We will build it in two versions — WITHOUT the hooks (broken), then WITH the hooks (fixed) — so you can see exactly what changes.



## Version 1 — WITHOUT Hooks (Broken) ■

Paste this into your app and open the browser console. Every `console.log` tells you exactly what is re-rendering.

```
import { useState } from 'react';  
// ████████████████████████████████████████████████████████████████████████████  
// ProductItem – renders ONE product row  
// ████████████████████████████████████████████████████████████████████████████  
function ProductItem({ product, onRemove }) {  
  console.log('ProductItem rendered:', product.name);  
  // ■ TEST: open console and click 'Toggle Theme'  
  // You will see EVERY ProductItem log fires  
  // even though products didn't change at all!  
  return (  
    {product.name} – ■{product.price}  
    onRemove(product.id)>  
    Remove  
  );  
}  
  
// ████████████████████████████████████████████████████████████████████████████  
// ProductList – renders all products  
// ████████████████████████████████████████████████████████████████████████████  
function ProductList({ products, onRemove }) {  
  console.log('ProductList rendered');  
  // ■ useMemo missing – filter runs on EVERY render  
  // Even toggling theme recalculates this!  
  const expensive = products.filter(p => p.price > 100);  
  return expensive.map(p =>  
  );  
}  
  
// ████████████████████████████████████████████████████████████████████████████  
// App – root component  
// ████████████████████████████████████████████████████████████████████████████  
export default function App() {  
  const [theme, setTheme] = useState('light');  
  const [products, setProducts] = useState([  
    { id: 1, name: 'Phone', price: 999 },  
    { id: 2, name: 'Book', price: 50 },  
    { id: 3, name: 'Laptop', price: 1500 },  
  ]);  
  
  // ■ New function created every render  
  // Toggling theme = new handleRemove = ProductList re-renders  
  const handleRemove = (id) => {  
    setProducts(prev => prev.filter(p => p.id !== id));  
  };  
}
```

```
return (  
  setTheme(t => t === 'light' ? 'dark' : 'light')}>  
  Toggle Theme  
);  
}
```

■ **HOW TO TEST:** Open browser console → click 'Toggle Theme' button You will see: *ProductList rendered + all ProductItem rendered logs fire Even though products never changed — pure wasted work* ■

## Version 2 — WITH Hooks (Fixed) ■

Same app. Three targeted changes — `React.memo`, `useMemo`, `useCallback`. Each change has a comment explaining exactly why it's there.

```
import { useState, useMemo, useCallback, memo } from 'react';  
// ████████████████████████████████████████████████████████████████████████████  
// FIX 1: Wrap ProductItem in React.memo  
// ████████████████████████████████████████████████████████████████████████████  
// WHY: stops re-render when parent re-renders with same props  
// TEST: click Toggle Theme → this log should NOT fire  
// WITHOUT IT: fires on every theme toggle even though  
// product and onRemove didn't change  
const ProductItem = memo(function ProductItem({ product, onRemove }) {  
  console.log('ProductItem rendered:', product.name);  
  return (  
    {product.name} – Rs.{product.price}  
    onRemove(product.id)}>Remove  
  );  
});  
// ████████████████████████████████████████████████████████████████████████████  
// ProductList component  
// ████████████████████████████████████████████████████████████████████████████  
function ProductList({ products, onRemove }) {  
  console.log('ProductList rendered');  
  // FIX 2: useMemo for expensive filter  
  // WHY: filter only runs when products array changes  
  // TEST: click Toggle Theme → 'filtering...' should NOT log  
  // WITHOUT IT: filter runs on every render including theme toggle  
  const expensive = useMemo(() => {  
    console.log('filtering products...');  
    return products.filter(p => p.price > 100);  
  }, [products]); // only re-run when products changes  
  return expensive.map(p =>  
  
  );  
}  
// ████████████████████████████████████████████████████████████████████████████  
// App – root component  
// ████████████████████████████████████████████████████████████████████████████  
export default function App() {  
  const [theme, setTheme] = useState('light');  
  const [products, setProducts] = useState([  
    { id: 1, name: 'Phone', price: 999 },  
    { id: 2, name: 'Book', price: 50 },  
    { id: 3, name: 'Laptop', price: 1500 },  
  ]);  
  // FIX 3: useCallback for handleRemove  
  // WHY: same function reference across renders  
  // React.memo on ProductItem checks onRemove prop  
  // without useCallback → new function → memo fails  
  // TEST: click Toggle Theme → ProductItem logs should NOT fire  
  // WITHOUT IT: new handleRemove ref → memo sees changed prop → re-renders  
  const handleRemove = useCallback((id) => {  
    setProducts(prev => prev.filter(p => p.id !== id));  
  }, []); // [] = never recreate (setProducts is always stable)
```

```
return (  
  setTheme(t => t === 'light' ? 'dark' : 'light')}>  
  Toggle Theme  
);  
}
```

## How to Test — Step by Step

### Step 1: Run Version 1 (without hooks)

Paste Version 1 code into App.jsx

Open browser → F12 → Console tab

Click 'Toggle Theme' button

■ **What you see in console: ProductList rendered ProductItem rendered: Phone ProductItem rendered: Laptop filtering products...** → All 4 logs fire on every theme toggle. Products never changed!

### Step 2: Run Version 2 (with hooks)

Paste Version 2 code into App.jsx

Open browser → F12 → Console tab

Click 'Toggle Theme' button

■ **What you see in console: (nothing)** → Zero logs on theme toggle. All 3 hooks protected the re-renders. → Only 'filtering products...' fires when you click Remove (products changed)

### Step 3: Test each hook individually — remove one at a time

Remove React.memo from ProductItem → theme toggle re-renders all items

Remove useMemo from filter → 'filtering products...' logs on every theme toggle

Remove useCallback from handleRemove → React.memo on ProductItem stops working

## What Breaks Without Each Hook

| Remove this                 | What breaks                                                                                   | How to see it in console                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| React.memo on ProductItem   | ProductItem re-renders on every parent re-render even with same props                         | Toggle theme → 'ProductItem rendered: Phone' + 'ProductItem rendered: Laptop' fire |
| useMemo on filter           | Filter runs on every render — expensive computation wasted on unrelated state changes         | Toggle theme → 'filtering products...' fires even though products didn't change    |
| useCallback on handleRemove | New function ref every render → React.memo on ProductItem sees changed prop → memo is useless | Toggle theme → all ProductItem logs fire (memo defeated by unstable function ref)  |
| All three together          | Every state change re-renders everything — no optimisation at all                             | Toggle theme → ALL console logs fire on every single click                         |

## One Line Summary

| Hook               | Job in this example                                               | Without it                                                   |
|--------------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| <b>React.memo</b>  | ProductItem skips re-render when product + onRemove didn't change | Re-renders all items on every theme toggle                   |
| <b>useMemo</b>     | Filter only recalculates when products array actually changes     | Filter runs on every render including theme changes          |
| <b>useCallback</b> | handleRemove stays the same reference so React.memo can work      | New function ref defeats React.memo — items re-render anyway |

■ **Interview tip:** When asked about these hooks, always explain them together. 'React.memo checks if props changed. useMemo stabilizes object/array props. useCallback stabilizes function props. All three need each other to work.'